

Especificação Técnica: Inventory Service

Domínio: Core Domain | Persistência Principal: Redis (In-Memory Data Grid)

Objetivo Arquitetural: O *Inventory Service* é o componente mais crítico em cenários de alta concorrência. A sua única responsabilidade é garantir que não ocorrem duplas vendas de um mesmo lugar (*overbooking*). Para assegurar latência de milissegundos durante picos de acesso, utiliza o **Redis** para gerir os bloqueios temporários com expiração automática (TTL).

1. Enums de Domínio

Definem os estados possíveis para um lugar dentro do ciclo de vida da compra.

```
package inventory_service.domain.enums;

public enum SeatStatus {
    AVAILABLE, // Lugar livre para compra
    LOCKED,    // Bloqueado temporariamente no carrinho de um utilizador
    SOLD       // Venda confirmada e paga
}
```

2. Entidade de Persistência (Redis Hash)

A entidade principal não é guardada num banco relacional tradicional durante a fase de checkout, mas sim no Redis, aproveitando o tempo de vida (TTL) nativo.

```

package inventory_service.domain.entities;

import org.springframework.data.annotation.Id;
import org.springframework.data.redis.core.RedisHash;
import org.springframework.data.redis.core.TimeToLive;
import org.springframework.data.redis.core.index.Indexed;
import inventory_service.domain.enums.SeatStatus;
import java.util.UUID;

@RedisHash("SeatLock")
public class SeatLock {

    @Id
    private String lockId; // UUID gerado para a tentativa de reserva

    @Indexed
    private UUID eventId;

    @Indexed
    private UUID sectorId;

    private UUID userId;
    private Integer quantity; // Utilizado caso o setor seja não-numerado (ex: Geral)
    private String seatNumber; // Utilizado caso seja cadeira numerada (ex: "B-42")

    private SeatStatus status;

    @TimeToLive
    private Long ttl; // Tempo em segundos para o bloqueio expirar (ex: 600s = 10 min)

    // Construtores, Getters e Setters omitidos por brevidade
}

```

3. DTOs (Records de API REST)

Estes objetos são utilizados caso o *Checkout Service* precise de consultar o estado do inventário de forma síncrona antes de submeter o pedido.

```

package inventory_service.api.dtos;

import inventory_service.domain.enums.SeatStatus;
import java.time.LocalDateTime;
import java.util.UUID;

public record SeatReservationRequest(
    UUID eventId,
    UUID sectorId,
    Integer quantity,
    String seatNumber // Pode ser nulo se o setor não tiver lugares marcados
) {}

public record SeatStatusResponse(
    String lockId,
    SeatStatus status,
    LocalDateTime expiresAt // Indica ao frontend quando o carrinho expira
) {}

```

4. Contratos de Mensageria (RabbitMQ)

A comunicação principal de bloqueio e liberação ocorre de forma assíncrona. Estes registos (Records) devem residir na Biblioteca Partilhada (Shared Library).

```
package matchpass_common.events.inventory;

import java.util.UUID;

// Publicado pelo Inventory quando consegue garantir o lugar
public record SeatReservedEvent(
    UUID orderId,
    String lockId,
    UUID userId
) {}

// Publicado pelo Inventory quando o lugar já está ocupado por outro utilizador
public record SeatReservationFailedEvent(
    UUID orderId,
    String reason
) {}
```

5. Repositório (Spring Data Redis)

A interface que comunica com o Redis para persistir as chaves e procurar bloqueios existentes.

```
package inventory_service.infrastructure.repositories;

import inventory_service.domain.entities.SeatLock;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.UUID;

@Repository
public interface SeatLockRepository extends CrudRepository<SeatLock, String> {
    // Permite procurar se um lugar específico já está bloqueado num evento
    List<SeatLock> findByEventIdAndSectorIdAndSeatNumber(UUID eventId, UUID sectorId, String
seatNumber);
}
```

6. Métodos de Serviço Core (Lógica de Negócio)

A classe `InventoryService` contém a lógica para aplicar as travas, verificar disponibilidade e consumir os eventos do *Checkout* e *Payment*.

```

package inventory_service.application.services;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class InventoryManagementService {

    private final SeatLockRepository repository;
    private final RabbitTemplate rabbitTemplate;

    // Construtor com injeção de dependências...

    /**
     * Tenta aplicar um bloqueio. Acionado via RabbitMQ (OrderPlacedEvent).
     */
    public void tryLockSeat(OrderPlacedEvent event) {
        // 1. Verificar se o lugar já existe no Redis (está bloqueado?)
        // 2. Se livre, criar novo SeatLock com TTL de 10 minutos
        // 3. Guardar no Redis via repository.save()
        // 4. Publicar SeatReservedEvent para o Payment Service prosseguir
        // 5. Se ocupado, publicar SeatReservationFailedEvent para cancelar a ordem
    }

    /**
     * Confirma a venda definitiva. Acionado via RabbitMQ (PaymentApprovedEvent).
     */
    public void confirmSeatSold(PaymentApprovedEvent event) {
        // 1. Procurar o SeatLock no Redis
        // 2. Alterar o status para SOLD
        // 3. Remover o TTL para que o registo não expire
        // (Opcional: Mover o registo consolidado para uma base relacional secundária de
        histórico)
    }

    /**
     * Liberta o lugar imediatamente. Acionado via RabbitMQ (PaymentRefusedEvent).
     */
    public void releaseSeat(PaymentRefusedEvent event) {
        // 1. Procurar o SeatLock no Redis
        // 2. Remover do Redis (repository.delete), libertando o lugar para outros utilizadores
    }
}

```